

Sistemas de Big Data

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



**Sistemas de
Big_Data**

Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

3

Capítulo

Una panorámica a los Sistemas de Big Data

Ricardo García Ródenas

El proceso de procesamiento de la información deriva una estructura multicapa para los sistemas BD como la mostrada en la figura 3.1. La capa de datos recolecta los datos provenientes de fuentes diversas como redes sociales, sensores o de la propia organización. La siguiente capa es la del almacenamiento, donde se integra y se almacena en bases de datos (relacionales o NoSQL). En la siguiente capa es la de análisis en la que se procesa los datos. Finalmente existe una capa de visualización que es la que muestra los resultados al usuario.

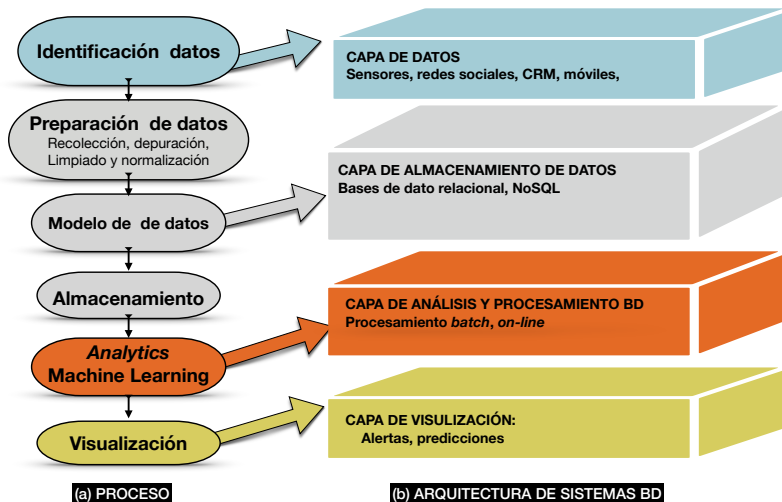


Figura 3.1: Procesos y estructura multicapa en los sistemas Big Data.

En este capítulo se abordan tres aspectos de los sistemas BD. El primero es dar una panorámica de diversas herramientas disponibles para la construcción de un sistema BD, el segundo es dar aplicaciones de estos sistemas y el tercero elaborar un proyecto basado en estas tecnologías.

3.1. Arquitectura de sistemas Big Data

La implementación de un sistema BD conduce a un ecosistema de aplicaciones y a una arquitectura como mostrada en al Figura 3.2. Sus elementos fundamentales son:

1. Una **aplicación web** donde los usuarios pueden elegir dentro de un catálogo de servicios.
2. Una **aplicación de ingestión de datos** que está diseñada para extraer los registros y procesarlos.
3. Una **aplicación de procesamiento** que funciona como el motor de la arquitectura.
4. Una **aplicación de aprendizaje automático** para obtener la información útil para el usuario.
5. Un **motor de búsqueda** para extraer el análisis de nuestros datos.

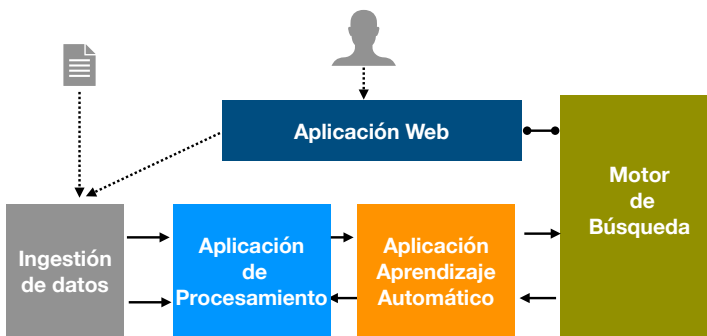


Figura 3.2: Arquitectura para los sistemas Big Data.

3.1.1. Tipología sistemas Big Data

Una arquitectura de un sistemas BD como la mostrada en la figura 3.2 puede ser de tres tipos en función de la disponibilidad de los datos y de los requerimientos en la respuesta del sistema:

- **Procesamiento batch** o por **lotes**. En estos sistemas BD los datos están todos disponibles y almacenados antes de iniciar el procesamiento. Cuando se produce la ejecución del programa, debido al elevado coste computacional, ya no se produce ninguna iteración con el usuario hasta la obtención del resultado final.
- **Procesamiento streaming**. Este tipo de procesamiento y análisis de datos se basan en la implementación de un modelo de flujo de datos en el que los datos asociados a series de tiempo (hechos) fluyen continuamente a través del sistema BD (ver 3.2). Estos sistemas BD no tienen limitaciones de tiempo en el procesamiento del flujo de datos, ni tampoco existe una exigencia en cuanto al tiempo de generación del *output* por cada input recibido en el sistema.
- **Procesamiento en tiempo real**. En este tipo de procesamiento se aborda flujo de datos pero la diferencia con el procesamiento *streaming* es que si existe limitaciones sobre el tiempo de respuesta.

3.1.2. Implementación en la nube

Con la popularidad de los sistemas *cloud*, los sistemas BD tienen la posibilidad de ser desplegados en la nube. La nube ofrece servicios de computación y almacenamiento, de tal forma que el usuario pueda disponer de estos recursos hardware-software sin la necesidad de adquirirlos. La posibilidad de alquilar estos recursos de hardware en vez de comprarlos e instalarlos en el centro de procesamiento de datos (CPD) de la organización presenta ventajas en cuanto ahorro de costes, flexibilidad, ahorro de tiempo de instalación, escalabilidad, información en tiempo real y actualizaciones periódica pero presenta una dependencia de los proveedores y de la conexión a internet. Este tipo de soluciones *cloud* presentan los siguientes retos fundamentales que deben acometerse por el propietario y usuario de la infraestructura:

1. *Hackeo*. Este término hace referencia a las formas de obtener, copiar, leer o ver información confidencial llevadas a cabo por personas no autorizadas. Estas acciones se realizan con herramientas sofisticadas que actúan durante el proceso de intercambio de información dentro de redes inseguras. El *hackeo* es un desafío global en el que numerosas organizaciones gastan millones de euros/dólares para proteger sus infraestructuras. Muchos de los servicios BD están empleando sistemas criptográficos para evitar ataques y poder preservar un nivel de funcionalidad para los usuarios.
2. Protección de datos. Este aspecto hace referencia a la divulgación de información no autorizada debido a protocolos deficientes o mal uso de los sistemas de información. Incurrir en estos fallos tiene como consecuencia una pérdida de reputación para las organizaciones.

3. *Cuellos de botella en el procesamiento.* Con la popularidad de computación en la nube, uno de los desafíos clave para los proveedores de servicios en la nube es poder atender tantos servicios como los demandados por los usuarios. Una de las demandas de los clientes es tener la mejor accesibilidad a los servicios en cualquier momento. Si el sistema no funciona como lo esperado se denomina cuello de botella en el procesamiento. El cuello de botella se produce por un incremento del número de procesos realizados en un servidor. Los proveedores de estos servicios deben gestionar estas situaciones desplegando más servidores, replicando la información y el middleware entre los servidores.
4. *Escalabilidad.* Este aspecto hace referencia a la capacidad del sistema de poder estar disponible cuando se produce un incremento esperado de la demanda. Alcanzar esta característica en los sistemas de BD pasa por el uso de aplicaciones software que usen algoritmos eficientes (con baja complejidad computacional) y ejecuciones concurrentes de las tareas que involucren, además de tener la capacidad de expandir la infraestructura cloud.
5. *Concurrencia.* La concurrencia es la ejecución simultánea de varios procesos que comparten recursos. El reto es no tener tiempos muertos entre procesos, lo requiere estrategias óptimas de sincronización.
6. *Tolerancia a fallos.* Esta característica de los sistemas BD requiere la detección de fallos de una manera controlada de tal forma que se pueda interrumpir durante el menor tiempo posible las componentes con un funcionamiento erróneo. El objetivo es minimizar las pérdidas de datos. Este reto requiere una gestión eficiente de los recursos y de los mecanismos de control, para incorporar y liberar recursos de una forma óptima.
7. *Disponibilidad.* Los usuarios demandan tener acceso a los servicios en cualquier momento y desde cualquier lugar. Para los proveedores de servicios es esencial que exista disponible al menos una máquina que pueda ejecutar un determinado tipo de servicios. Si un servidor se cae, es necesario que el sistema funcione con los recursos restantes. Los recursos críticos deben ser replicados para asegurar un determinado nivel de servicio.

3.2. Procesamiento batch de BD: El marco MapReduce

Un sistema BD con un procesamiento batch posee una arquitectura de computación distribuida donde los datos se recolectan y almacenan antes de que alimenten la capa de análisis. La infraestructura para la computación puede ser un *cluster* que está formado por multitud de ordenadores con un *hardware* similar y conectados en una red local o una red de ordenadores distribuidos geográficamente y con un *hardware* heterogéneo. Estos sistemas llegan a procesar grandes cantidades de datos que pueden llegar a tamaños de *petabytes*.

Hasta el momento, el marco conceptual canónico para el procesamiento batch de BD es el denominado *MapReduce* que fue introducido inicialmente por Google en 2004 como respuesta al problema de poder indexar la web al ritmo al que ésta iba creciendo. Esta solución se basaba en un sistema de archivos distribuidos, que le proporcionaba la capacidad de poder procesar en paralelo grandes cantidades de información. Google propuso que su *MapReduce* marco contuviese tres módulos: una máquina para ejecutar MapReduce, un sistema de ficheros distribuidos llamado *Google File System* (GFS) y iii) una base de datos NoSQL distribuida llamada *BigTable*. Tras este anuncio la fundación *Apache Software Foundation* comenzó la implantación en código abierto de *MapReduce*. Los módulos fundamentales de este proyecto son *Hadoop MapReduce* y *Hadoop YARN* como motores de ejecución y *Hadoop Distributed File System* (HDFS) y *Hbase* como sustitutos de *BigTable*.

Las plataformas BD con procesamiento batch más difundida son Apache Hadoop y Apache Spark. Los principios básicos sobre los que se asientan estas plataforma para poder procesar grandes cantidades de datos son: i) operaciones paralelas sobre trozos de datos y ii) inmutabilidad de los datos.

Las requisitos fundamentales de un procesamiento batch son i) un modelo de programación, ii) una escalabilidad automática y lineal, iii) ser tolerante a fallos, iv) no tener pérdidas de datos y v) poder procesar todo (o casi) el conjunto histórico de datos.

MapReduce es un modelo de programación distribuido. Los registros se abstraen como una estructura clave-valor, $\langle k, v \rangle$ que permiten almacenarlos y administrar sus operaciones. El valor v puede contener cualquier tipo de dato.

La implementación distribuida del esquema *MapReduce* se realiza con una arquitectura master-esclavo como se muestra en la Figura 3.3. Los datos de entrada se almacenan en un sistema de ficheros distribuidos y se particionan en subconjuntos que alimentan a los diferentes nodos esclavos. MapReduce consta de tres pasos:

Map: En esta etapa cada nodo esclavo realiza un primer procesamiento con los datos locales, y escribe el resultado en un almacenamiento temporal. Hay copias redundantes de los datos pero el nodo maestro se encarga que solo se procese una copia de cada dato.

Shuffle: Los nodos esclavos redistribuyen los datos basándose en las claves de salida k_2 , de forma que todos los datos que pertenecen a una clave k_2 se encuentran en el mismo nodo esclavo.

Reduce: En la última etapa los nodos esclavo procesan ahora cada grupo de datos de salida, por clave.

La etapa *Shuffle* es realizada de forma automática por MapReduce. El usuario solo tiene que definir las funciones de computación de la primera y tercera etapa: $map(k_1, v_1)$ y $reduce(k_2, list(v_2))$. Para cada input clave-valor $\langle k_1, v_1 \rangle$, la función $map(k_1, v_1)$ devuelve un output vacío o claves-valor intermedias $\langle k_2, v_2 \rangle$.

Los valores intermedios se agrupan de acuerdo al valor de la claves k_2 para generar los pares $\langle k_2, list(v_2) \rangle$. La función *reduce* toma $\langle k_2, list(v_2) \rangle$ y genera el conjunto final de clave-valor $\langle k_3, v_3 \rangle$. Las claves k_1, k_2 y k_3 y sus valores v_1, v_2 y v_3 pueden consistir en cualquier tipo de dato.

Lo primero es que una copia del código de las funciones *map* y *reduce* es enviado a los diferentes procesadores esclavos. Esta operación es fundamental para no *mover* los datos y *acercar* la computación a ellos. Entonces el ordenador maestro asigna tareas *map* o *reduce* entre los diferentes procesadores. Cada procesador esclavo con la tarea *map* lee sus correspondientes datos y ejecuta sobre ellos la función *map* y escribe los resultados en ficheros intermedios. Cuando finaliza esta primera fase *map*, los procesadores asociados a la fase *reduce* leen estos ficheros intermedios ya clasificados y ejecutan sobre ellos las tareas *reduce*, escribiendo los ficheros con el output final.

La ejecución de la fase *map* constan de varias tareas: i) *read* para la lectura de los datos de los ficheros distribuidos, ii) *map* para la ejecución de la función *map* definida por el usuarios, iii) *collect* para particionar y almacenar los outputs de la ejecución de las funciones *map*, iv) *spill* para ordenar, combinar y escribir los resultados en un fichero en un disco local. Posteriormente se ejecuta la fase *shuffle* para transferir (copiar) los resultados de las operaciones *map* a los nodos *reduce*. Las tareas *reduce* consta de las etapas: i) *merge* para combinar los outputs ordenados de *map*, ii) *reduce* para ejecutar la función *reduce* definida por el usuario y iii) *write* para escribir el output final en el sistemas de ficheros distribuidos. Notar que la fase *reduce* puede comenzar antes de estar terminada la fase *map* para reducir el tiempo de ejecución. Se podría ir ejecutando nuevas tareas *reduce* cada vez que una nueva clave k se ha identificado.

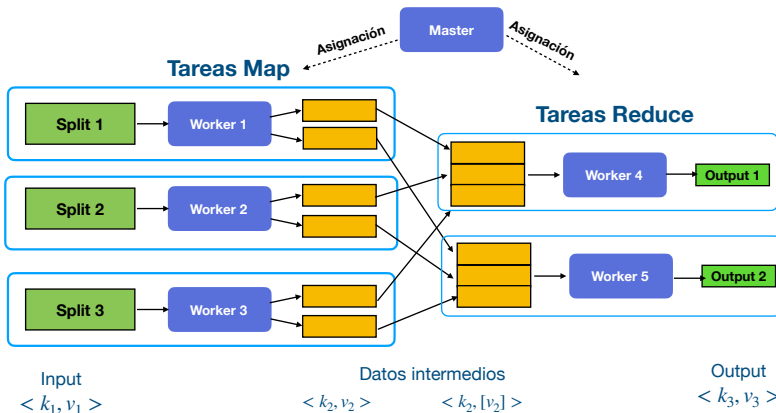


Figura 3.3: Esquema MapReduce.



Ejemplo MapReduce Considérese que dada una lista de comentarios de usuarios, queremos determinar la primera y la última vez que un usuario realizó un comentario y el número total de comentarios de ese usuario. La clave de MapReduce es mantener los datos donde están y mover la computación a los datos. Toda la computación se realiza a través de las funciones *map* y *reduce*. El primer paso es que el ordenador maestro hace copias de la función *map* en un conjunto de servidores. Cada ordenador de forma local lee los atributos de los ficheros XML y construye las clave-valor $\langle k, v \rangle$ donde k es el id de un usuario y v es la 3-tupla $[\text{mínimo}, \text{máximo}, \text{cuenta}]$. La función *map* rellena los valores *mínimo* y *máximo* con la fecha de creación de los comentarios y pone el valor *cuenta* = 1 indicando que el usuario ha realizado un comentario. La función *reduce* itera a través de los valores clave-valor de un usuario para encontrar las fechas mínimas y máximas, y sumar el número de comentarios. El mínimo se va calculando como el mínimo entre el mínimo actual y el del valor mínimo de la tupla actual. La misma lógica se aplica al máximo, pero utilizando un operador mayor que. El recuento de cada valor se agrega a una suma corriente. Después de determinar las fechas mínimas y máximas de todos los valores de entrada, el recuento final se establece en nuestro valor de salida. La clave de entrada se escribe entonces en el sistema de archivos junto con el valor de salida, esto es, el $\langle k_3, v_3 \rangle$.

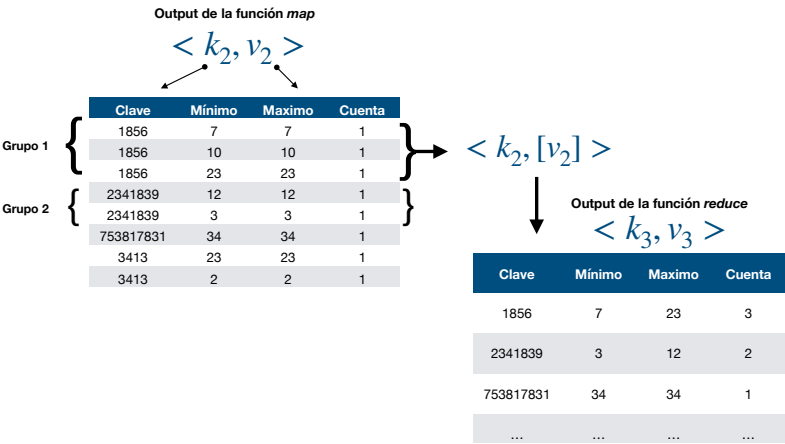


Figura 3.4: Ejemplo de MapReduce.

3.3. Procesamiento stream de BD

En este sistema BD los datos llegan continuamente a la capa de análisis y son procesados inmediatamente. Estos sistemas BD no tienen limitaciones de tiempo en el procesamiento del flujo de datos, ni tampoco existe una exigencia en cuanto al tiempo de generación del output por cada input recibido en el sistema. Por ejemplo,

un sistema diseñado para el recuento de *tweets* conteniendo ciertas palabras clave es un sistema procesamiento stream donde los datos se generan dinámicamente y en ciertos momento de alta intensidad de tráfico la respuesta se puede ralentizar e incluso el sistema, si fuese necesario, puede descartar un número de *tweets*. Los sistemas BD *streaming* deben disponer de suficiente memoria para almacenar entradas en cola, además la tasa media de procesamiento debe ser mayor que la tasa media de entrada al sistema para que la cola de procesamiento (necesidades de almacenamiento) no creciese hasta el infinito.

Ejemplos de este tipo de plataformas BD son Apache Storm, Flink o Spark Streaming.

3.4. Procesamiento en tiempo real BD

Los sistemas BD en tiempo real, a diferencia del procesamiento stream, si imponen limitaciones al tiempo de respuesta, que en muchos casos es inferior a unos pocos segundos.

Una cuestión esencial a analizar en un sistema BD en tiempo real es que consecuencias tiene el sobrepasamiento de la capacidad del sistema BD. Si el sistema BD pierde la entrada puede hacer caso omiso de la pérdida y seguir procesando sin detenerse. En según qué aplicaciones, esto no supone un problema crítico pero en otros sistemas esto no es admisible. Por ejemplo, un sistema de control del tráfico ferroviario no puede en ningún momento perder el control del sistema por motivos de seguridad.

Las técnicas de procesamiento en tiempo real se pueden agrupar en dos categorías: i) *computación en memoria* cuyo objetivo de estas técnica es reducir el coste de ejecución de sistemas basados en *MapReduce*, consiguiendo ejecución de ciertos trabajos en menos de segundos y ii) *consultas en tiempo real* sobre datos estructurados y no estructurados.

La lentitud de ciertos sistemas basados en un esquema *MapReduce* proviene por dos hechos fundamentales: i) los procedimientos para el inicio de la ejecución de los procesos no están optimizados y ii) se almacenan grandes bloques de datos sobre discos duros. La tecnología actual permite tasas de lectura sobre estos dispositivos entre 100 y 200 megabytes por segundo. Esto conduce a que si 1 terabytes es almacenado en 20 máquinas, una simple búsqueda consumirá un orden de magnitud de minutos en lugar de segundos. Los principales dispositivos en memoria procesan 10 gigabytes por segundo, que es significativamente superior a los 200 megabytes de los discos duros. Además el tiempo de latencia en el acceso es de nanosegundos frente a los milisegundo del disco duro. El precio del RAM también es asequible, menos de 20000 dolares un TB.

Se debe poner de manifiesto que la *computación en memoria* no significa que la base de datos completa sea almacenda en memoria. Estos sistemas almacenan en memoria los datos que son usados frecuentemente en el trabajo iterativo

Las consultas en tiempo real en sistemas BD hace referencia a un funcionamiento interactivo, permitiendo respuestas en segundos o minutos. En procesamiento batch se admiten respuestas en horas o días. La primera solución en este campo fue *Dremel* desarrollada por Google. Esta herramienta para conseguir realizar consultas en tiempo real sobre BD usa dos técnicas fundamentales: i) un formato novedoso columnas para estructuras anidadas y ii) un algoritmo de agregación escalable para computación paralela.



Ejercicio. Supóngase que cada hora se suben 1000 fotografías a una plataforma con una resolución tal que se requieren 3 MB/fotografía para su almacenamiento. El procesamiento de cada imagen consume 30 s. Supóngase que el único coste computacional es el de lectura y el de procesamiento. Si se adopta un procesamiento batch con un esquema MapReduce con 20 máquinas, estima el tiempo necesario para poder procesar las imágenes generadas en un día. Estima también el número mínimo de máquinas necesarias para poder implementar un sistema con procesamiento streaming.

3.5. Plataformas para sistemas BD

A continuación se reseñan las tecnologías más difundidas en la implementación de sistemas BD. Estas tecnologías, ver figura 3.5, cuenta con menos de tres lustros de existencia.



Figura 3.5: Soluciones para implementar sistemas BD.

3.5.1. Apache Hadoop

Apache Hadoop es una de las tecnologías mas populares para el procesamiento batch de BD. Es una plataforma de software de código abierto para almacenar datos y ejecutar aplicaciones en un cluster de ordenadores.

Hadoop tiene varias fortalezas, provenientes del marco conceptual *MapReduce*, como su escalabilidad aproximadamente lineal o su capacidad de tolerancia a fallos. *Hadoop* implementa el módulo *Hadoop distributed file system* (HDFS) que es esencial para la escalabilidad del sistema. HDFS es un sistema de archivos distribuidos que permite de forma transparente a los programadores almacenar un archivo de datos en diferentes máquinas de un cluster de Hadoop. Cada fichero que se almacena en HDFS se divide en bloques y es cada uno de estos bloques lo que se replica en diferentes nodos del cluster. HDFS permite escalar de una máquina a miles de ellas.

A la vez que se ha ido desarrollando *Hadoop* se ha creado un ecosistema alrededor suyo con productos como *Apache Hive*, *Apache Pig*, *Apache Zookeeper*, *Apache Sqoop*, entre otros. Por ejemplo, *Apache Sqoop* permite interactuar entre las tradicionales bases de datos relacionales con *Hadoop*. *Hadoop Sqoop* permite leer datos desde una base de datos relacional y escribir dichos datos en HDFS y viceversa. Además de poder emplear base de datos relacionales en lugar de emplear HDFS, existen plug-in para la entrada y salida de datos que permiten usar bases de datos NoSQL como *HBase* o *Cassandra*, *HBase* permite almacenar grandes cantidades de datos estructurados o usar *Mahout* como herramienta de aprendizaje automático.

Además tiene otras características propias como definición de tareas mas complejas empleando *YARN* (*Yet Another Resource Negotiator*). El módulo *YARN* desacopla la gestión de los recursos de la planificación de las tareas. Por un lado, el denominado *ResourceManager* se encarga de la gestión de los recursos a nivel global dentro del cluster y el *ApplicationMaster* planifica las tareas por cada una de las aplicaciones que se ejecutan dentro del cluster.

Algunas de las limitaciones de *Hadoop* son:

- *Bajo rendimiento para la computación en línea e iterativa.* *Hadoop* no permite la recursividad ni procedimientos interactivos. Su concepción *batch* hace que todos los datos deban estar preparados antes de iniciar el trabajo. El coste computacional para iniciar un proceso, copiando los códigos y planificando la ejecución, lo limita para un uso iterativo o de consultas en tiempo real. Existen en el mercado extensiones del marco *MapReduce* que permiten iteración y recursión como *Twister* o *HaLoop*.
- *Bajo rendimiento de computación.* El uso intensivo que hace del disco duro relentiza su procesamiento y además su diseño es inadecuado para el cálculo en memoria.
- *Baja capacidad de intercomunicación.*

3.5.2. Apache Spark

Apache Spark es una framework de programación de código abierto usado para el procesamiento batch, streaming y tiempo real de datos distribuidos. *Apache Spark* permite cargar los datos en memoria y consultarlos repetidamente, lo que lo convierte en una herramienta muy adecuada para el procesamiento iterativo y en línea (especialmente para los algoritmos de aprendizaje automático). El desarrollo de *Apache Spark* fue motivado por las limitaciones del paradigma *MapReduce/Hadoop* que obliga a seguir un flujo de datos lineal y con un uso intensivo del disco. *Apache Spark*, debido a su computación en memoria, tiene una gran velocidad en el procesamiento de datos con respecto al marco *Apache Hadoop*. Las tareas ejecutadas con *Hadoop* duran horas o incluso días (dependiendo del volumen de datos a tratar y de la complejidad del algoritmo) mientras que este tiempo es reducido drásticamente si se emplea *Spark*.

Cuando se oye hablar de *Spark* puede tratarse de dos cosas: el motor Spark, también conocido como *Spark Core* (ver figura 3.6), o el proyecto de código abierto *Apache Spark* (ver figura 3.7), que es un término *paraguas* para Spark Core y las aplicaciones Spark que lo acompañan, es decir, aplicaciones como *Spark SQL* (para procesamiento de datos estructurado), *Spark Streaming* (para el procesamiento de datos en *streaming*), *Spark MLlib* (para la ejecución iterativa de métodos de aprendizaje automático), o *Spark GraphX* (para el procesamiento gráfico). Las diferentes APIs para Python, Scala, Java, SQL o R le permiten a los programadores de *Spark* eliminar la complejidad que requiere *Hadoop*. Esto acelera la velocidad de la construcción de aplicaciones.

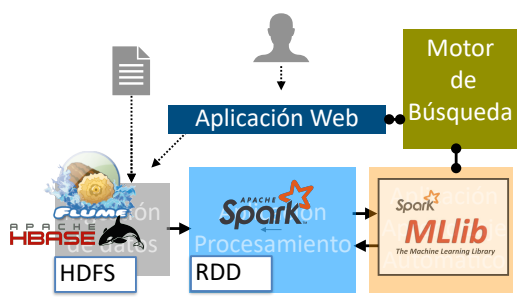


Figura 3.6: Motor Spark en una arquitectura BD.

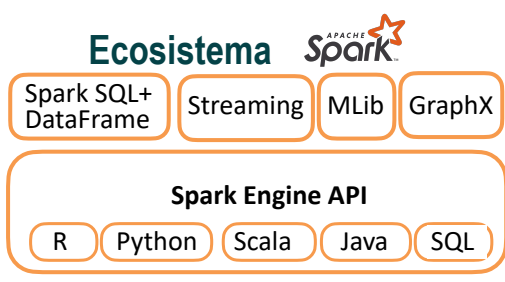


Figura 3.7: Ecosistema Spark.

Spark emplea una abstracción para definir la estructura distribuida de los datos denominada *Resilient Distributed Datasets* (RDD) que pueden ser considerada como estructuras de datos paralelos e inmutables. Los RDDs son tolerantes a fallos ya que se preservan información sobre sus padres y se puede recomponer la partición en caso de fallo. Hay dos formas de crear RDDs: paralelizando una colección existente en el programa principal, o referenciando un conjunto de datos en un sistema de almacenamiento externo, como HDFS.

Los RDDs están diseñados para contener enormes cantidades de datos, que no caben en una sola máquina por lo tanto, los datos tienen que ser particionados a través de múltiples máquinas/nodos. Una partición es la unidad atómica (indivisible) de paralelismo en Apache Spark que se almacena en un nodo del cluster. Los RDDs en Apache Spark son una colección de particiones. Spark particiona automáticamente los RDDs y distribuye las particiones entre diferentes nodos. Las operaciones a realizar sobre los RDDs colocan automáticamente las tareas en las particiones, manteniendo la ubicación de los datos persistentes. Los resultados intermedios pueden persistir en la memoria o en el disco para ser reutilizados. También se puede personalizar la particiones para optimizar la colocación de los datos.

La figura 3.8 muestra como *Spark Streaming* procesa los datos en *streaming*. El primer paso es dividir el flujo de datos en porciones, denominados *micro batch*. Estos *micro batch* se transforman en un conjunto de RDDs denominados *Dstream* (*stream* discretizado) que es procesado. *Spark* lee la información de la fuente a intervalos de tiempo fijo (tiene un periodo mínimo de 0.5 segundos).



Figura 3.8: Procesamiento streaming con Spark.

Otro acierto de *Apache Spark* es su posibilidad para poder ejecutarse sobre distintas plataformas, entre ellas la de YARN de *Hadoop*, posibilitando el procesamiento *streaming* que *Hadoop* carece de forma nativa. En este caso *Spark* tiene la posibilidad de generar RDD desde diversas fuentes de datos como *HDFS* y *HBase*. Hay que elegir cuidadosamente la plataforma donde ejecutar *Apache Spark* ya que configurar, desplegar, monitorizar, escalar e introducir protocolos de seguridad en un cluster de *Apache Spark* puede llegar a ser un proceso complejo.

Como resumen, las principales **diferencias** entre *Hadoop* y *Spark* son:

- *Spark* hace los cálculos basados en la memoria mientras que *Hadoop* hace uso intensivo del disco duro.
- *Hadoop* se basa en el marco MapReduce, por lo que cada tarea se basa en las etapas *Map* y/o *Reduce*. Es trabajo del programador definir cada una de ellas y encadenarlas para implementar la aplicación. Con *Spark*, el propio motor crea complejas cadenas de estas etapas a partir de la lógica de la aplicación. Esto permite a los desarrolladores expresar algoritmos complejos y *pipelines* de procesamiento de datos dentro del mismo trabajo y permite al marco optimizar el trabajo en su conjunto, lo que conduce a un mejor rendimiento.

Mientras que las **características comunes**:

- Cada nodo del cluster tiene un acceso local a los datos.
- Ejecución por etapas *Map / Reduce* (etapas separadas por fases de *Shuffle*).
- Dependencia del sistema de archivos distribuido para la persistencia en disco (HDFS)

3.5.3. Otras tecnologías

Storm fue creada por Twitter para el procesamiento stream en BD. Este entorno se ejecuta sobre una máquina virtual de java. Este entorno permite programar rápidamente aplicaciones que procesen grandes flujos de datos. Un programa en Storm viene definido por dos abstracciones: los *spouts* y los *bolts*. Un *spout* es una fuente de stream. Los *spouts* pueden leer los datos desde una cola o incluso generarlos ellos mismos. Un proceso *bolt* emplea uno o varios *streams* de entrada y produce un *stream* de salida. La mayoría de los procesos lógicos se expresan en los *bolts*. Cada programa en Storm es un grafo de *spouts* y *boults* denominado Topología. Storm tiene tres nodos que son el nodo maestro, denominado *nimbus*, los nodos *workers* y *zookeeper*. En nodo maestro distribuye y coordina la ejecución de la topología mientras que los *workers* son los responsables de la ejecución de los *spouts/boults*, *zookeeper* sincroniza la computación distribuida.

Esta herramienta al igual que Spark procesa los datos en tiempo real y tiene sus pros y contras respecto a ella.

S4. Es un sistema para el procesamiento stream distribuido, inspirado por el marco *MapReduce* para procesar stream de datos. Está implementado por Yahoo mediante Java. El stream de datos alimenta a S4 como eventos.

Kafka permite construir una *pipeline* de fuentes de datos en tiempo real e integrarla en *Spark*. Esta herramienta recibe los datos desde *kafka* y los procesa en tiempo real. Esta herramietta no solo es útil para procesamiento en tiempo real, si no también para desarrollar aplicaciones de procesamiento streaming.

Apache Flink. Es una solución de código abierto que permite analizar tanto datos en modo batch como en tiempo real. La programación de *flink* y *MapReduce* comparten muchas similitudes. Esta plataforma permite un procesamiento iterativo y computación en tiempo real sobre un stream de datos recolectado mediante herramientas como *flume* o *Kafka*. *Apache flink* cuenta con *FlinkML* que es una librería de aprendizaje automático que cuenta con muchos algoritmos muy eficientes y escalables para aplicaciones BD.

Flume es un servicio distribuido empleado para la recolección de datos en tiempo real, con almacenamiento temporal que permite mover eficientemente grandes cantidades de datos.

Plataforma	Tipo procesamiento	Latencia	Desarrollador	Flujo principal
Hadoop	batch	Minutos o más	Yahoo	claves de acceso
Storm	stream	subsegundos	Twitter	Tuplas
Spark	batch/stream	pocos segundos	Berkley AMPLay	DStream
S4	stream	pocos segundos	Yahoo	Eventos
Flink	batch/stream	pocos segundos	Apache Software Foundation	claves de acceso

Tabla 3.1: Plataformas para BD

Desde el punto de vista de la arquitectura, hay que elegir plataformas que posean capacidades de escalabilidad horizontal (incrementar el número de nodos del cluster), monitorización, planificación de tareas, recuperación automática de errores, etc. Estas plataformas deberían abstraer los recursos físicos (el hardware propiamente dicho) y permitir ver la plataforma como un conjunto de recursos y no como una colección de máquinas.

Hoy en día una solución para el despliegue de sistemas BD es lo que se denomina *Big Data as a Service* (BDaaS). Esta solución integra la plataforma BD, el software que interactúa con la plataforma y el propio hardware como un único servicio para el usuario. Las ventajas de emplear una solución BDaaS es un soporte técnico altamente cualificado, la fácil monitorización del servicio, las actualizaciones automáticas de la tecnología, seguridad a nivel de plataforma, la escalabilidad y alta disponibilidad de la plataforma.

Amazon Kinesis es un servicio BDaaS de procesamiento e incorporación de datos. Es capaz de tratar cientos de miles de datos e incluirlos de manera continua en un stream. Los outputs generados por Kinesis pueden ser empleados en el mismo orden que son generados con las aplicaciones de aprendizaje automático de Amazon.

La tabla 3.1 resume las características de las plataformas para abordar problemas BD.

3.6. Aplicaciones de los sistemas BD

A nivel internacional y para diversos sectores económicos existen múltiples casos de éxito en la aplicación de sistemas BD. Es especialmente importante analizar las posibles aplicaciones de esta tecnología situándonos en nuestro contexto nacional. El hecho más relevante a corto y mediano plazo es el *Plan de Digitalización de PYMEs 2021-2025* que contempla una inversión pública de 4.656 millones de euros con un impacto directo en forma de ayudas a la digitalización de 4.459 millones de euros.

El informe *Digital Economy and Society Index* en 2018 recoge que la penetración de la tecnología BD es del 12 % de las empresas con 10 o más empleados de la UE-27, mientras que es del 11 % en España. De todas las tecnologías clave en el ámbito de la digitalización empresarial, tal y como señala el informe ePyme 2019,

la analítica en BD es la menos utilizada por las empresas españolas. Además, la tendencia, según cifras del INE, es decreciente, pasando del 11, 24 % en 2018, a un 8, 47 % en 2020. Existen grandes diferencias en la tendencia en función del tamaño de las empresas.

El Plan recoge 5 Ejes de Actuación estructurados en 16 medidas. Desde nuestro punto de vista las medidas que más impacto pueden tener en la aplicación de las tecnología BD son:

1. *Medida 1 Programa Digital Toolkit*, dentro del eje 1) Digitalización Básica para las PYMEs. sí, el Plan contempla una inversión pública significativa, de 1000 millones de euros en el periodo 2021-2023 para poner en marcha un programa horizontal de gran escala para incorporar a las PYMEs un paquete de herramientas de digitalización (toolkit) ya disponibles (herramientas habilitadoras, formación, página web, sistema de gestión de recursos ERP, sistema de gestión de clientes CRM, venta por internet, sistema de marketing digital, ...). Más que la tecnologías BD, debido al tamaño de las PYMEs y por tanto a su volumen de negocio, las herramientas que tiene un impacto son la analítica de datos. Los CRM y el comercio electrónico son fuentes generadoras de datos del negocio cuyo análisis puede mejorar la eficacia y eficiencia de los procesos de la empresa y con ello la cuenta de resultados.
2. *Medida 8 Programa de Innovación Disruptiva para la Transformación Digital en PYMEs*, dentro del eje 3) Innovación Disruptiva y emprendimiento digital. El Plan contempla acciones para impulsar los modelos más complejos e innovadores, de mayor riesgo, basados en la integración de soluciones más disruptivas y el uso intensivo de datos, donde el internet de las cosas, los servicios cloud, o la inteligencia artificial, deben jugar un papel protagonista. Este es un campo idóneo para buscar soluciones BD innovadoras, guiadas por la creatividad. Sectores diana son la industria, el turismo y el comercio. La inversión se implementará a través de convocatorias de ayudas (subvenciones) en concurrencia competitiva.
3. *Medida 6 Formación para Expertos en Transformación Digital de las PYMEs*, dentro del eje 2) Apoyo a la Gestión del Cambio. El objetivo de este programa es formar a jóvenes Expertos en digitalización de las PYMEs como agentes del cambio en los procesos de transformación digital.
4. *Medida 7 Programa de Agentes del Cambio*. objetivo de este programa es financiar la incorporación de profesionales en transformación digital en las PYMEs para desarrollar su plan de digitalización y acelerar su transformación digital, actuando como agente del cambio dentro de la empresa. Los agentes del cambio podrán haberse formado en el Programa de Expertos en Transformación Digital. Tiene previsto invertir 300 millones de euros en el periodo 2021-2023.

Las aplicaciones de BD dependen fuertemente del tamaño de la empresa o institución que la implementa. En este sentido es conveniente realizar una división de las aplicaciones atendiendo a esta consideración. Primero analizamos aplicaciones potenciales del BD que se pueden llevar a cabo en PYMEs y posteriormente damos ejemplos desarrolladas en grandes empresas.

En el proceso de digitalización de una PYME cabe destacar las siguientes herramientas:

- ERP, (acrónimo del inglés *Enterprise Resource Planning*) es un conjunto de aplicaciones de software integradas, que permiten automatizar las prácticas de negocio relacionadas con los aspectos operativos o productivos de la empresa.
- CRM, (acrónimo de *Customer Relationship Management*) es una aplicación que permite centralizar en una única Base de Datos todas las interacciones entre una empresa y sus clientes. Esta herramienta recopila la información histórica de las interacciones de los clientes con la empresa. El objetivo es gestionar las acciones comerciales (captación de clientes, fidelización, etc.) a partir de un cuadro de mando detallado.
- Plataformas de venta online. Este sistema de compra y venta de productos y servicios emplea Internet como medio principal de intercambio, permitiendo acceder a más clientes, no tener horarios y reducir los costes. Los aspectos negativos son la falta de confianza de los clientes, productos y servicios no tangibles, competencia, dificultades técnicas en su despliegue y requieren un tiempo para obtener resultados.

Estas herramientas centralizan la información áreas como compras, producción, logística, finanzas, recursos humanos, marketing, servicios, proyectos y atención al cliente. Por tanto son una fuente generadora de datos para la empresa. Un segundo aspecto esencial a todas ellas es su modularidad y escalabilidad que permiten ir implementando las funcionalidades que se vayan necesitando, reduciendo así las necesidades de inversión. Desde nuestro punto de vista las aplicaciones BD, (más data que big en el contexto de las PYMEs) es la de incorporar la *inteligencia y aprendizaje de la experiencia* en estas funcionalidades. A modo de ejemplo se listan alguna de estas posibilidades:

1. Incremento del comercio usando información de la propia empresa.

- a) Búsqueda de nuevas oportunidades de venta a través de segmentación de clientes y venta cruzada de productos
- b) Análisis de clientes e historial de transacción, para identificar clientes potenciales para la contratación de servicios o productos complementarios, o que puedan contratar servicios de mayor valor.
- c) Segmentación de cliente para afinar mensajes de marketing y proporcionar ofertas específicas.

- d) Introducción en los CRM de estrategias de personalización de cliente para establecer lazos más estrechos con clientes, lo cual mejora la lealtad.

2. Incremento del comercio usando información externa a la empresa.

- a) Análisis de datos de navegación web y hábitos de consumo.
- b) Marketing que explota las redes sociales para maximizar la difusión de los productos y servicios.

3. Mejoras de Procesos.

- a) Automatización y control de procesos.
- b) Introducción de filtros inteligentes para la mejora del comercio electrónico.
- c) Cuadro de Mandos en tiempo real, la información siempre está disponible sin esperas de actualización de los datos
- d) Diseño de productos o servicios.
- e) Análisis de proveedores.

Las grandes corporaciones son las que tienen las mayores posibilidades para la aplicación de tecnologías BD. Disponen de grandes cantidades de datos y disponen los recursos materiales y humanos para acometer estos proyectos. Estas aplicaciones constituyen los mayores casos de éxito, por su envergadura, pero no son extrapolables a empresas de pequeña escala. El valor que tienen es que delimitan las fronteras de la tecnología y sirven para crear nuevas soluciones a problemas relevantes pero de menor escala. Estimulan la creatividad humana y es el valor que deben tener para el desarrollo. A continuación listamos algunos de estos ejemplos:

- *AMAZON* es la empresa líder en el comercio electrónico. Esta empresa emplea tecnologías BD para recolectar, analizar y utilizar la cantidad masiva de datos a la que tienen acceso a partir del historial de búsqueda y de compra de un usuario. Esta tecnologías las aplica en la recomendación de productos, optimización de la cadena de suministro, optimización de precios y detección de fraudes.
- *NETFLIX* ha desarrollado uno de los sistemas de recomendación más elaborado que le permite predecir qué quiere ver una persona, y por tanto conocer qué tipo de series o películas debe producir. Para ello recolecta y analiza enormes cantidades de datos relacionados al comportamiento del usuario: los géneros que busca, el número de capítulos que ve seguidos, el tiempo que se demora escogiendo algo que ver, etc.
- *APPLE* emplea técnicas de BD para la fidelización de sus clientes a sus productos. Los datos los recogen a través de sus dispositivos portátiles como el iWatch así como las diversas apps desarrolladas.

- **COCA-COLA** es la compañía de refrescos más grande del mundo. Ha empleado técnicas BD para el desarrollo de nuevos productos, como el sabor *Cherry Sprite* que nació del análisis de los datos recolectados de las máquinas dispensadoras de gaseosa. Esta empresa también realiza análisis de sentimientos en redes sociales para tener una imagen clara de cómo se perciben sus productos en los diferentes países, qué tipo de personas los consumen, en dónde, en qué circunstancias. También han empleado técnicas de IA para mejorar sus plantas de embotellamiento hasta el funcionamiento de sus máquinas expendedoras.
- **STARBUCKS** es una empresa de red de cafeterías que emplea técnicas BD para determinar la tasa de éxito de un nuevo establecimiento en función de su ubicación, tráfico y comportamiento de los usuarios. Esta información le permite optimizar las localizaciones para sus nuevas tiendas.

3.7. Elaboración proyectos BD

A veces alguien tiene una idea, se enamora de ella y trata de sacarla adelante cueste lo que cueste. Se debe hacer el ejercicio de mirarla fríamente, sin pasión cegadora, este es el objetivo que persigue la metodología planteada en esta sección.

Todo proyecto empresarial, como son la implantación de una solución BD, debe ser evaluado racionalmente. Para ello los responsables de la organización deben poder contestar a cuestiones fundamentales como ¿Por qué debe la empresa llevar a cabo un nuevo proyecto? ¿Qué ocurriría si no se lleva a cabo? ¿Qué beneficios se pueden obtener de él? ¿Merece la pena asumir los costes previstos? ¿Cuanto tiempo requiere el proyecto? Una herramienta esencial para que los responsables puedan aprobar o rechazar un proyecto es el llamado *business case*.

Un *business case* responde a las anteriores preguntas. Argumenta por qué el proyecto sería beneficioso basándose en los costes, beneficios y riesgos que tendría la nueva inversión. El beneficio puede ser económico, pero también puede tener en cuenta otros criterios, como reducción de la carga de trabajo o el aumento de la satisfacción de los empleados/clientes.

Un *business case* es un informe detallado del proyecto. Debe recoger el por qué es necesario el nuevo proyecto, qué beneficios y costes se esperan. La argumentación se debe apoyar en cifras y por tanto se debe especificar las fuentes de datos en las que se apoyan, por ejemplo, información financiera interna, estudios de mercados relacionados, etc. El *business case* también debe examinar las posibles alternativas al proyecto y los riesgos asociados. No existe una plantilla estandarizada, sin embargo atendiendo a la metodología de gestión de proyectos PRINCE2, un documento *business case* debe constar de las siguientes secciones:

- **Portada:** Debe contener un título que debe ser expresivo y el/los autor/es. Dado que el documento pasa por muchas manos y siempre está siendo revisado o corregido, se debe incluir una tabla para recoger quien y cuando revisa y/o aprueba la **versión** del documento.

- **Objetivos y alcance:** En esta sección se debe determinar de forma clara, sencilla y concreta los objetivos que se intentarán alcanzar, especificando todas las cosas que se deben realizar para alcanzarlos (alcance).
- **Resumen ejecutivo:** Esta sección debe estar escrita para que los responsables de aprobar/rechazar el proyecto puedan tener una primera evaluación (rápida) del mismo. Esta sección es la última que se escribe y debe explicar los problemas que debe resolver el proyecto, qué condiciones se han tenido en cuenta, qué beneficios y retorno de la inversión se esperan, cuántos recursos se necesitan para completar el proyecto y cómo es el resultado deseado.
- **Motivación:** Todo proyecto debe estar enmarcado dentro del plan estratégico de la organización. Los principales motivos son: i) cumplir requisitos regulatorios, legales o sociales; ii) nueva oportunidad estratégica de negocio; iii) nueva oportunidad debido a avances tecnológicos; iv) nuevos productos / servicios demandados por el mercado o v) mejora de productos/servicios existentes.
- **Opciones de negocio:** Un proyecto representa una posible solución a un problema. Existe otras alternativas, según PRINCE2, siempre hay que considerar tres opciones: i) no hacer nada, ii) hacer lo mínimo y iii) hacer algo. Los escenarios ii) y iii) son los que se deben analizar en el informe contestando en esta sección con medidas cuantitativas y cualitativas: ¿qué consecuencias monetarias conllevan estas opciones? ¿Y qué otros efectos que no pueden expresarse en cifras se producirán?
- **Beneficios esperados:** El beneficio esperado del proyecto tiene que ser medible, de modo que los resultados puedan ser contrastables tras la finalización. Esta sección debe incluir la lista de beneficios con razones y valores de referencia, así como el instante en el que se espera obtener dicho beneficio. Por ejemplo, un proyecto para una empresa de mensajería puede contener dos tipos de beneficios: reducción los costes de distribución en un 10 % y reducción los tiempos de entrega en un 15 % tras el remplazamiento del 24 % de la flota.
- **Desventajas esperadas:** Listado de efectos secundarios negativos derivados de la implantación del proyecto. Esto no es un riesgo (fenómenos negativos que pueden ocurrir con cierta probabilidad) sino consecuencias negativas que se observarán.
- **Calendario:** Se detallará el inicio y el final del proyecto y los hitos claves como cuando se espera pagar el proyecto y se inicie los beneficios del mismo. Por este motivo puede que el intervalo temporal analizado vaya más allá de la duración real del proyecto.
- **Coste:** Esta sección contiene todos los costes del proyecto durante su ejecución y tras su finalización, por ejemplo, los costes de mantenimiento, costes de las actualizaciones, etc.

- **Valoración de la inversión:** En esta sección se analiza el valor del proyecto como inversión. La base del cálculo está formada por los datos de las secciones sobre costes y beneficios incluidos en el documento.

Alguno indicadores básicos que se incluyen son:

VAN (Valor Actual Neto) corresponde al valor presente de los flujos de caja netos originados por una inversión. Esto es, una inversión amortizada en un periodo de varios años se producen costes y beneficios en años diferentes, donde el valor del dinero varía por efecto de la inflación. El índice VAN homogeniza todos estos flujos (cantidad/año) al valor actual del dinero. Por ejemplo, tener en 2021 un beneficio de 1000 € no es equivalente que tener el mismo beneficio diez años después.

$$VAN = -I_0 + \sum_{t=1}^n \frac{F_t}{(1+k)^t}$$

donde I_0 es la inversión inicial (por eso el signo negativo), F_t representa el flujo de caja en el año t , n es el número de periodos considerados y k es la tasa de descuento. Si el valor de $VAN > 0$ el proyecto genera valor, en otro caso la inversión pierde valor.

TIR (Tasa Interna de Retorno) es la tasa de descuento que hace que la inversión tenga un valor de $VAN = 0$, esto significa que bajo dicha tasa la inversión es neutra (ni pierde ni genera valor). Para calcularla se debe resolver la siguiente ecuación

$$-I_0 + \sum_{t=1}^n \frac{F_t}{(1+TIR)^t} = 0,$$

siendo la incógnita TIR . Notar que si el TIR del proyecto es menor que el interés pagado en productos financieros indicaría que como inversión el proyecto es peor que contratar dichos productos financieros.

Periodo de recuperación es el número de años m que la empresa necesita para recuperar la inversión. Tendríamos que calcular el menor valor de m (años) que cumple la siguiente ecuación:

$$-I_0 + \sum_{t=1}^m \frac{F_t}{(1+k)^t} > 0,$$

ROI (Return on Investment) es una medida de rentabilidad que se utiliza para evaluar el rendimiento de una inversión y permite comparar diferentes proyectos. Por ejemplo, un proyecto puede reportar un valor VAN superior a otro pero su rentabilidad dependerá de la inversión inicial I_0 . Este índice quiere estandarizar este aspecto para poderlos

comparar. Este índice se define por:

$$ROI = \frac{VAN}{I_0}$$

- **Principales riesgos:** Resumen de los mayores riesgos para el proyecto y su probabilidad de ocurrencia, así como las contramedidas a tomar en caso de aparición. Aquí basta con una visión general para ser evaluados en la toma de decisión de la aceptación / rechazo del proyecto; los detalles adicionales sobre los riesgos se registran en otros documentos.

Valoración de inversiones. Supóngase que se realiza un proyecto de duración $n = 3$ años con una inversión inicial de $I_0 = 25000$ €. La tasa de descuento para este periodo es de $k = 5\%$. Supóngase que los flujos de caja son

Año	Costes	Ingresos
1	5000	3000
2	7500	20000
3	8000	35000

• **VAN** Lo primero es calcular los flujos netos de caja como la resta de ingresos menos costes:

Año	Costes	Ingresos	F_t
1	5000	3000	-2000
2	7500	20000	12500
3	8000	35000	27000

Sustituyendo en la fórmula del VAN las anteriores cantidades y teniendo en cuenta que $k = 5/100 = 0,05$, obtenemos:



$$VAN = -25000 + \frac{-2000}{(1 + 0,05)} + \frac{12500}{(1 + 0,05)^2} + \frac{27000}{(1 + 0,05)^3} = 7756,7$$

• **ROI.** Calculamos

$$ROI = \frac{VAN}{I_0} = \frac{7756,7}{25000} = 0,3103$$

Como la tasa se expresa en porcentaje, el ROI es de 31,03 %. Esta tasa expresa que la inversión tiene una rentabilidad (en tres años) de un 31 %.

• **TIR.** Notar que el ROI no tiene en cuenta el tiempo empleado en la obtención de las ganancias de la empresa. Una estimación burda es dividir el ROI por el número de años pero esta estimación lineal (regla de tres) no captura la no linealidad del problema. El TIR intenta resolver esta cuestión. Esta tasa puede ser vista como el interés anual que debería dar una entidad bancaria por depositar la inversión inicial I_0 para conseguir el mismo beneficio que el proyecto. Como hay que resolver una ecuación no lineal recurrimos al uso del ordenador. El código 3.1 permite calcular el TIR para el ejemplo. El valor obtenido es $TIR = 0,1577$. Este valor es aproximado y se puede comprobar que cumple aproximadamente la ecuación

$$-25000 + \frac{-2000}{(1 + 0,1577)} + \frac{12500}{(1 + 0,1577)^2} + \frac{27000}{(1 + 0,1577)^3} = -0,0209 \approx 0€$$

Listado 3.1: Cálculo de la TIR en python

```
1 # Importamos la libreria numpy para calcular la TIR
2 # Tasa Interna de Retorno. En inglés se denomina IRR (Internal Rate of Return).
3 import numpy as np
4 # Flujos netos. El primer elemento del vector es la inversion inicial (negativo)
```

```
5 flujos=[-25000,-2000,12500,27000]  
6 print (round(np.irr(flujos), 5))
```



Ejercicio. Elabora un esbozo de *business case* para un proyecto BD titulado: Análisis de sentimientos de los usuarios de la operadora ferroviaria TRENRA-PID.

Bibliografía

- [HFG⁺19] Dirk Helbing, Bruno S Frey, Gerd Gigerenzer, Ernst Hafen, Michael Hagner, Yvonne Hofstetter, Jeroen van den Hoven, Roberto V Zicari, and Andrej Zwitter. *Will Democracy Survive Big Data and Artificial Intelligence?*, pages 73–98. Springer International Publishing, Cham, 2019.
- [McD09] John H. McDonald. *Handbook of Biological Statistics*. Sparky House Publishing, 2009.
- [WSM20] Martin Wiener, Carol Saunders, and Marco Marabelli. Big-data business models: A critical literature review and multiperspective research framework. *Journal of Information Technology*, 35(1):66–91, 2020.